

Promptathon

Transform weak, vague prompts into powerful, structured prompts using the ICEPOT Framework

Scenario 1: Search Functionality

- Scenario Name: Search Functionality
 - Weak Prompt: Test the search feature.
 - What I Feel is Missing:
 - Application type
 - Search criteria
 - Filters and sorting
 - Invalid and empty searches
 - Performance expectations
1. I – Instruction:
Generate manual test cases for the search functionality.
 2. C – Context:
The application is an e-commerce website that allows users to search for products by name, category, and brand. Users can also apply filters and sorting options.
 3. E – Example:
Verify that searching for "iPhone" displays relevant products containing the keyword.
 4. P – Persona:
Act as a Senior Manual QA Engineer.
 5. O – Output Format:
Provide test cases in a table with Test Case ID, Scenario, Steps, and Expected Result.
 6. T – Tone:
Technical and professional.

Rewritten Prompt:

Act as a Senior Manual QA Engineer. Generate manual test cases for the search functionality of an e-commerce website. Users can search products by name, category, and brand, and can apply filters and sorting options. Include positive, negative, empty search, partial keyword, filter, sort, and performance-related scenarios. For example, verify that searching for "iPhone" displays relevant products. Present the output in a

table with Test Case ID, Scenario, Steps, and Expected Result. Use a professional and technical tone.

Observations:

1. The weak prompt did not generate any test cases and instead asked for additional tool permissions. The rewritten ICEPOT prompt clearly defined the application type, testing scope, and expected output format, which helped the AI generate detailed and usable test cases.
2. The improved prompt covered multiple test scenarios such as positive, negative, empty search, filter, sort, boundary, and performance testing. The weak prompt did not provide enough information for the AI to understand what aspects of the search feature needed to be tested.
3. The output from the ICEPOT prompt was structured in a test case table with steps and expected results, making it easier for a QA tester to execute and track testing activities. The weak prompt produced no actionable testing output.

Scenario 2: File Upload

- Scenario Name:File Upload
 - Weak Prompt: Generate test cases for file upload.
 - What I Feel is Missing:
 - Supported file types
 - File size limits
 - Success and failure conditions
 - Security validations
 - Error handling
1. I – Instruction:
Generate manual test cases for file upload functionality.
 2. C – Context:
The application allows users to upload profile documents. Supported file types are PDF, JPG, and PNG with a maximum size of 5 MB.
 3. E – Example:
Verify that a valid PDF file below 5 MB is uploaded successfully.
 4. P – Persona:
Act as a Senior Manual QA Engineer.
 5. O – Output Format:
Provide test cases in a table with Test Case ID, Scenario, Steps, and Expected Result.

6. T – Tone:
Technical and professional.

✓ Rewritten Prompt:

Act as a Senior Manual QA Engineer. Generate manual test cases for a file upload feature where users can upload PDF, JPG, and PNG files up to 5 MB. Include positive, invalid file type, oversized file, empty upload, duplicate upload, error message, and basic security scenarios such as uploading a malicious executable file. For example, verify that a valid PDF file below 5 MB uploads successfully. Present the output in a table with Test Case ID, Scenario, Steps, and Expected Result. Use a professional and technical tone.

Observations:

1. The ICEPOT prompt provided specific application requirements such as supported file types (PDF, JPG, PNG) and the 5 MB file size limit. Because of this, the generated test cases were more precise and aligned with realistic business requirements.
2. The improved prompt included detailed steps and expected results for every test case, making the output easier to execute during manual testing. The weak prompt mostly listed scenarios without enough execution detail.
3. The ICEPOT output focused on the exact scope requested, including boundary testing, validation checks, duplicate uploads, and security-related scenarios. This made the test cases more structured and directly applicable to a real project.

Scenario 3: Shopping Cart

- Scenario Name: Shopping Cart
 - Weak Prompt: Test the cart functionality.
 - What I Feel is Missing:
 - Add/remove/update actions
 - Quantity validation
 - Price calculations
 - Discount handling
 - Stock validation
1. I – Instruction:
Generate manual test cases for shopping cart functionality.

2. C – Context:
The application is an e-commerce website where users can add products to the cart, update quantities, remove items, and apply discount coupons.
3. E – Example:
Verify that the cart total updates correctly when the quantity of a product is increased.
4. P – Persona:
Act as a Senior Manual QA Engineer.
5. O – Output Format:
Provide test cases in a table with Test Case ID, Scenario, Steps, and Expected Result.
6. T – Tone:
Technical and professional.

Rewritten Prompt:

Act as a Senior Manual QA Engineer. Generate manual test cases for the shopping cart functionality of an e-commerce website. Users can add products, update quantities, remove items, and apply discount coupons. Include scenarios for price calculation, quantity updates, coupon validation, cart persistence, out-of-stock products, and item removal. For example, verify that the cart total updates correctly when the quantity of a product is increased. Present the output in a table with Test Case ID, Scenario, Steps, and Expected Result. Use a professional and technical tone.

Observations:

1. The rewritten prompt gave the output in a proper test case format, which made it easier to understand and use for manual testing activities.
2. The improved prompt generated more detailed scenarios around cart calculations, coupon validation, and stock handling, which are important business flows in an e-commerce application.
3. The output was more actionable because every test case included clear steps and expected results instead of only listing areas that should be tested.